



Universidad de Costa Rica
Sede del Atlántico
Recinto de Paraíso

IF-7100 Ingeniería del Software

Investigación:

Procesamiento masivo de datos

Profesor:

MSc. Leonardo Camacho

Integrantes:

Betzabeth Madrigal Gómez
José Pablo Ramírez Brenes
Karla Daniela Siles Siles

Junio, 2026

Tabla de contenidos

Introducción.....	2
Problema propuesto.....	3
Tecnologías implementadas.....	4
Características arquitectónicas priorizadas.....	8
Solución propuesta.....	9
Plan de pruebas.....	11
Resultados del plan de pruebas.....	18

Introducción

El procesamiento automatizado de grandes volúmenes de información es un desafío creciente en el ámbito tecnológico actual, especialmente cuando se requiere mantener altos estándares de velocidad, confiabilidad y capacidad de crecimiento. En este contexto, la gestión de correos electrónicos masivos representa un problema particularmente relevante para organizaciones que dependen de la comunicación digital como canal principal de interacción con sus usuarios.

El presente trabajo consiste en el diseño, desarrollo y evaluación de un sistema automatizado para el procesamiento de correos electrónicos a gran escala. Se implementó una solución técnica basada en una arquitectura modular que permite extraer, clasificar y responder mensajes de manera eficiente, cumpliendo con requisitos específicos de tiempo de respuesta y garantizando la integridad de la información procesada.

Como parte de la solución, se definieron las características arquitectónicas prioritarias, se seleccionaron las tecnologías más adecuadas y se ejecutó un plan de pruebas para validar el comportamiento del sistema. Los resultados obtenidos permiten concluir sobre la viabilidad y efectividad de la solución propuesta para entornos de producción con altas demandas de procesamiento.

Problema propuesto

1. Contexto

La empresa Daily Planet es un medio de comunicación con una gran cantidad de suscriptores a nivel mundial. Debido a su alcance, recibe diariamente un volumen muy alto de correos electrónicos provenientes de diferentes usuarios. Estos correos pueden tener diversos propósitos, como solicitudes de suscripción, consultas de información, envío de noticias o tips, y también quejas o reclamos.

2. Problema principal

El principal desafío de la empresa es que debe procesar todos los correos electrónicos sin perder ninguno y garantizar una respuesta adecuada para cada usuario. Además, el sistema debe ser capaz de manejar grandes volúmenes de información de forma continua, lo que hace necesario un enfoque altamente eficiente y escalable.

3. Necesidad del sistema

Se requiere el diseño de un sistema automatizado que pueda monitorear constantemente la llegada de nuevos correos, procesarlos y analizarlos para entender la intención del usuario. El sistema debe clasificar cada correo según su contenido y determinar si puede recibir una respuesta automática o si debe ser enviado a una cola de intervención posterior.

4. Requisitos de tiempo de respuesta

Uno de los requisitos más importantes es la velocidad de respuesta. El sistema debe enviar un acuse de recibo automático en un tiempo menor a 5 minutos después de recibir el correo.

5. Clasificación de correos

El sistema debe implementar un mecanismo de clasificación de correos, aunque el método no está definido. Se pueden utilizar enfoques como reglas basadas en palabras clave, sistemas basados en lógica o incluso integración con modelos de procesamiento de lenguaje natural (NLP) o inteligencia artificial. La elección del enfoque debe justificarse según las características arquitectónicas más importantes del sistema.

6. Simulación del sistema

Para efectos prácticos de la implementación, se debe desarrollar una aplicación que simule el envío masivo de correos electrónicos como carga de prueba. No es necesario utilizar una base de datos, ya que el enfoque está en el procesamiento de los correos. Los mensajes pueden ser gestionados mediante una cola de procesamiento, y no se contemplan archivos adjuntos en esta simulación.

Tecnologías implementadas

Para el desarrollo del sistema automatizado de procesamiento de correos electrónicos se utilizaron diferentes tecnologías orientadas al rendimiento, la fiabilidad, la escalabilidad y la mantenibilidad del proyecto.

Lenguaje de programación:

El lenguaje principal utilizado fue Python, debido a su simplicidad, flexibilidad y amplia disponibilidad de librerías enfocadas en automatización, procesamiento de datos y comunicación mediante protocolos de correo electrónico. Python permitió implementar de manera eficiente la lógica completa del sistema, incluyendo la lectura de correos mediante IMAP, el análisis y clasificación de mensajes, el envío automático de respuestas y la gestión de métricas de procesamiento.

Características principales:

- Simplicidad y flexibilidad para implementar lógicas complejas de automatización
- Amplia disponibilidad de librerías para protocolos de correo (IMAP, SMTP)
- Fácil integración con MongoDB y procesamiento de datos

Entornos de desarrollo:

Como entornos de desarrollo se utilizaron Visual Studio Code y PyCharm. Visual Studio Code fue utilizado principalmente por su ligereza, integración con terminales y facilidad para trabajar con múltiples servicios simultáneamente. Por otra parte, PyCharm aportó herramientas avanzadas de depuración, navegación de código y administración de proyectos Python, facilitando el mantenimiento y desarrollo del sistema.

Características principales:

Visual Studio Code

- Ligereza y rapidez para ejecución continua
- Integración nativa con terminales y control de versiones
- Amplio ecosistema de extensiones para Python

PyCharm

- Herramientas avanzadas de depuración paso a paso
- Navegación inteligente de código y refactorización
- Administración integrada de proyectos y entornos virtuales

La arquitectura implementada siguió el patrón ETL (Extract, Transform, Load), permitiendo dividir el procesamiento en etapas independientes y organizadas. En la fase Extract se desarrolló un primer backend encargado de conectarse al servidor IMAP de Gmail para leer y extraer los correos electrónicos entrantes. En la fase Transform se implementó un segundo backend responsable de analizar y clasificar los correos según categorías como “Suscripción”, “Queja”, “Solicitud de información”, “Envío de noticia” o “Atención humana”. Finalmente, en la fase Load se desarrolló un tercer backend encargado de actualizar la información procesada, almacenar los casos que requieren atención humana en un archivo CSV y enviar respuestas automáticas al remitente utilizando el protocolo SMTP. La separación en tres backends permitió mejorar la escalabilidad, organización y mantenibilidad del sistema, facilitando además la ejecución independiente de cada etapa del procesamiento.

Características principales:

Patron ETL (Extract, Transform, Load) - Arquitectura de procesamiento

- Separación en tres etapas independientes y organizadas
- Escalabilidad al permitir ejecutar cada fase por separado
- Mantenibilidad facilitada por la modularidad del diseño

Base de datos no relacional:

Para el almacenamiento de la información se utilizó MongoDB Atlas mediante un clúster web. La elección de MongoDB se debe a que los correos electrónicos representan información semi estructurada cuyos contenidos pueden variar considerablemente entre mensajes. MongoDB permitió almacenar documentos flexibles en formato JSON sin necesidad de esquemas rígidos, facilitando el crecimiento y adaptación futura del sistema. Además, el uso de MongoDB Atlas permitió contar con una base de datos accesible desde la nube, simplificando la comunicación entre los distintos backends y garantizando disponibilidad continua.

Características principales:

MongoDB Atlas - Base de datos no relacional en la nube

- Almacenamiento flexible de documentos JSON sin esquemas rígidos
- Accesibilidad desde la nube para comunicación entre backends
- Escalabilidad horizontal para manejar grandes volúmenes de datos

Estructura de la base de datos:

El registro mostrado corresponde a un documento individual dentro de una colección de una base de datos NoSQL (probablemente MongoDB). Cada documento representa un correo electrónico que ha sido extraído, procesado y clasificado automáticamente por el sistema.

A continuación, se describe cada campo del documento:

1. Identificadores y metadatos técnicos

Campo	Valor ejemplo	Descripción
_id	ObjectId('6a2639aca5235e0a62c19d9c')	Identificador único del documento, generado automáticamente por MongoDB.
id_mensaje_imap	"861"	Identificador asignado por el servidor IMAP de origen. Permite evitar el reprocesamiento del mismo correo.

2. Datos del correo original

Campo	Valor ejemplo	Descripción
Remitente	"beth2002@gmail.com"	Dirección de correo electrónico del remitente.
Asunto	"Quiero suscribirme al boletín #1"	Asunto del mensaje original.
Contenido	"Hola, deseo suscribirme al boletín de Daily Planet."	Cuerpo del mensaje.

3. Tiempos de procesamiento

Campo	Valor ejemplo	Descripción
fecha_extraccion	2026-06-08T03:40:28.782+00:00	Fecha y hora en que el sistema extrajo el correo del buzón IMAP.

tiempo_final	2026-06-08T03:40:41.117+00:00	Fecha y hora en que finalizó el procesamiento completo del correo.
tiempo_procesamiento	12.3351	Duración total del procesamiento en segundos (diferencia entre tiempo_final y fecha_extraccion).

4. Estados de procesamiento

Campo	Valor ejemplo	Descripción
leido	true	Indica si el correo ya fue leído por el sistema (evita duplicados).
procesado	true	Indica si el correo completó todas las etapas del pipeline de procesamiento.
clasificado	true	Indica si se asignó una categoría automática al correo.

5. Clasificación y acción tomada

Campo	Valor ejemplo	Descripción
categoria	"Suscripcion"	Categoría detectada por el sistema (ejemplo: Suscripción, Reclamo, Consulta).
destino	"Respuesta automática"	Acción determinada para este correo (respuesta predefinida, derivación a humano, etc.).
respuesta_enviada	true	Indica si el sistema ya envió una respuesta al remitente.

6. Control de errores

Campo	Valor ejemplo	Descripción
errores	Array (empty)	Lista de errores ocurridos durante el procesamiento. Al estar vacío, no hubo incidencias.

Almacenamiento estructurado por medio de CSV (Comma Separated Values)

Adicionalmente, se utilizó un archivo CSV para almacenar la cola de atención humana. Esta solución se implementó para registrar los correos clasificados como “Queja” o “Atención humana”, los cuales requieren revisión posterior por parte de operadores humanos. El formato CSV permitió mantener un registro ligero, portable y fácil de revisar manualmente, sin agregar complejidad innecesaria a la arquitectura.

Características principales:

- Almacenamiento estructurado
- Formato ligero y portable sin infraestructura adicional
- Fácil revisión manual por parte de operadores humanos
- Simplicidad de implementación sin complejidad innecesaria

HTML y CSS.

Se desarrolló un frontend sencillo para el simulador de envío de correos utilizando HTML y CSS con el objetivo de visualizar información básica del sistema y facilitar la interacción con algunos procesos del proyecto. HTML se utilizó para estructurar las interfaces de usuario, mientras que CSS permitió mejorar la presentación visual y organización de los elementos mostrados. Debido a que el enfoque principal del proyecto estaba orientado al procesamiento masivo de datos y automatización de correos, se optó por una interfaz minimalista enfocada principalmente en funcionalidad y simplicidad.

Características arquitectónicas priorizadas

- **Performance (Rendimiento)**

El rendimiento es la característica más crítica del sistema, ya que existe un requisito explícito de enviar un acuse de recibo en menos de 5 minutos. Esto implica que el sistema debe procesar los correos rápidamente desde el momento en que llegan.

Un bajo rendimiento afectaría directamente la experiencia del usuario y violaría los requisitos de tiempo establecidos.

- **Scalability (Escalabilidad)**

El sistema debe ser capaz de manejar un volumen muy alto de correos electrónicos provenientes de usuarios de todo el mundo. Además, la carga no es constante, por lo que el sistema debe escalar dinámicamente según la demanda.

La escalabilidad es esencial para garantizar que, aunque aumente significativamente el número de correos recibidos, el sistema siga respondiendo dentro de los tiempos establecidos.

- **Reliability (Confiabilidad)**

La confiabilidad es crucial porque el sistema no debe perder ningún correo electrónico. Cada mensaje recibido representa una interacción importante con un usuario, por lo que la pérdida de información afectaría directamente la calidad del servicio.

- **Maintainability (Mantenibilidad)**

La mantenibilidad es importante debido a que el sistema puede evolucionar con el tiempo, por ejemplo, mejorando el mecanismo de clasificación o agregando nuevas categorías de correos.

Solución propuesta

Proceso ETL (Extraer, Transformar, Carga)

El patrón ETL es un proceso fundamental en la gestión de datos que permite la organización e integración de diversas fuentes para ser extraídas, transformadas y cargadas en sistemas de destino para su posterior análisis o explotación.

Extracción:

Consiste en capturar los datos desde fuentes heterogéneas, como CRM, ERP o archivos planos, que a menudo tienen formatos incompatibles.

Transformación:

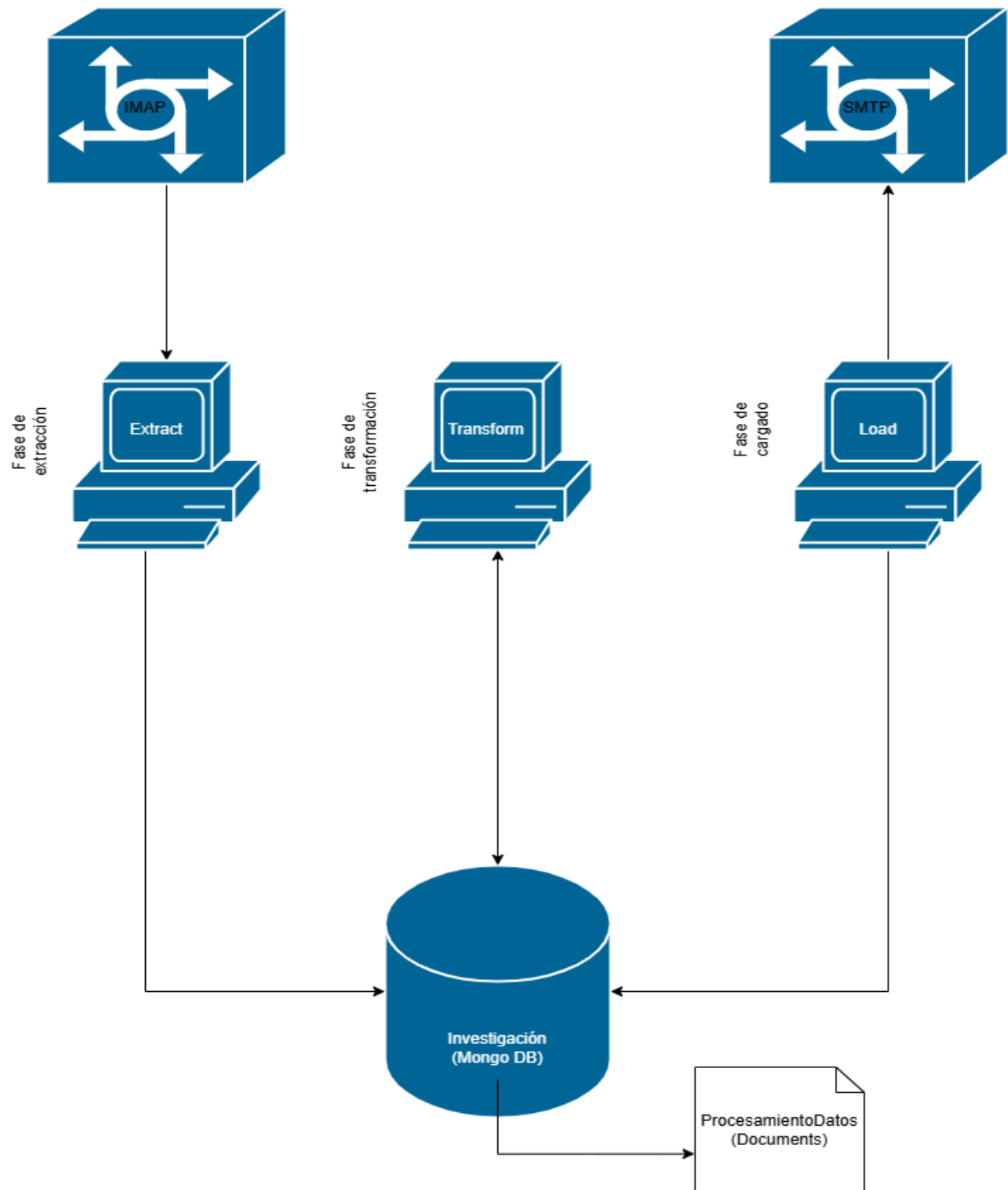
Es donde se produce la operación más crítica. La transformación modifica los datos en bruto para que presenten los formatos correctos.

Carga:

Implica mover los datos procesados al sistemas de destino. En efecto, esta última fase recolecta los datos extraídos, procesados, clasificados y llevados a este proceso que consiste en, finalmente, entregarlos al sistema de destino.

A continuación se muestra el diagrama de implementación del patrón de diseño ETL como solución al problema propuesto:

Patrón ETL (Extract, Transform, Load)



Plan de pruebas

El proyecto consiste en el desarrollo de un sistema automatizado de gestión de correos electrónicos para el Daily Planet, un medio de comunicación. El sistema esta compuesto por tres backends independientes que trabajan de forma coordinada:

- Backend 1 (Lectura): Se encarga de leer los correos electrónicos desde una cuenta de Gmail via protocolo IMAP, extraer la información relevante (remite, asunto y contenido) y almacenarla en una base de datos MongoDB Atlas para su posterior procesamiento.
- Backend 2 (Clasificación): Toma los correos almacenados en la base de datos, los analiza y clasifica en categorías como "Suscripción", "Queja", "Solicitud de información", "Envío de noticia" o "Atención humana".
- Backend 3 (Respuesta): Envía respuestas automáticas de confirmación a los remitentes una vez que los correos han sido clasificados.

Enfoque de pruebas

Las pruebas se diseñaron siguiendo dos enfoques complementarios:

Tipo de prueba	Objetivo	Aplicación
Pruebas unitarias	Validar el funcionamiento de componentes individuales de forma aislada	Cada funcion o metodo por separado
Pruebas de integración	Validar la interaccion entre componentes y con servicios externos	Flujo completo entre modulos y con BD

El flujo completo del sistema permite que los correos enviados por los usuarios sean procesados de manera automática, generando métricas de tiempo y rendimiento que permiten evaluar la eficiencia del sistema.

A continuación se describe el plan de pruebas implementado detallando los tipos de pruebas realizadas

Backend 1: Lectura y almacenaje en base de datos de los correos.

Prueba 1: Verificación de Conexiones

Tipo de prueba: Prueba de integración

Como se realizó: Se creó el archivo `test_conexiones.py` que contiene dos pruebas. La primera, `test_conexion_mongodb`, intenta establecer conexión con MongoDB Atlas utilizando la cadena de conexión proporcionada y ejecuta una operación de conteo para verificar que la conexión es funcional. La segunda, `test_conexion_imap`, intenta conectarse al servidor IMAP de Gmail (`imap.gmail.com` en el puerto 993) utilizando la librería `imaplib`, autenticándose con una contraseña de aplicación generada desde la configuración de seguridad de Google. Ambas pruebas verifican que los objetos de conexión no sean nulos y que las operaciones básicas respondan correctamente.

Prueba 2: Lectura de Correos desde el Buzon

Tipo de prueba: Prueba unitaria

Como se realizó: Se implemento el archivo `test_lectura_imap.py` que contiene tres pruebas. La primera, `test_leer_correos_no_leidos`, ejecuta el método `leer_correos_no_leidos()` de la clase `LectorIMAP` y verifica que el resultado sea una lista. La segunda, `test_estructura_correo`, toma el primer correo encontrado y valida que contenga los campos requeridos: `id`, `remitente`, `asunto` y `contenido`. La tercera, `test_limpiar_texto`, prueba la función auxiliar que elimina espacios en blanco y maneja valores nulos. Para estas pruebas, el método `setUp` establece la conexión IMAP antes de cada prueba y el método `tearDown` la cierra al finalizar.

Prueba 3: Guardado en Base de Datos

Tipo de prueba: Prueba unitaria

Como se realizó: Se creo el archivo `test_guardado_bd.py` que contiene dos pruebas. La primera, `test_guardar_correo`, utiliza el modelo `CorreoModel`.`crear_desde_imap()` para construir un documento con todos los campos requeridos (`remitente`, `asunto`, `contenido`, `id_mensaje_imap`, `fecha_extraccion`, `leido`, `tiempo_final`, `tiempo_procesamiento`, `procesado`, `clasificado`, `categoria`, `destino`, `respuesta_enviada` y `errores`), lo inserta en la colección "ProcesamientoDatos" de la base de datos "Investigación", y verifica que la operación retorna un ID válido y que el documento exista en la base de datos. La segunda, `test_contar_correos`, verifica que la función de conteo retorna un número entero válido. El método `tearDown` limpia los correos de prueba generados durante la ejecución.

Prueba 4: Prevención de Duplicados

Tipo de prueba: Prueba unitaria

Como se realizó: Se desarrolló el archivo `test_duplicados.py` que contiene una prueba llamada `test_prevision_duplicados`. Esta prueba crea dos correos con el mismo `id_mensaje_imap`,

guarda el primero y antes de intentar guardar el segundo verifica si ya existe un documento con ese identificador en la base de datos. La validación consiste en comprobar que el documento existe después del primer guardado, lo que demuestra que el segundo guardado no se ejecutaria gracias a la verificación previa implementada en el sistema. El método `tearDown` elimina los documentos de prueba.

Prueba 5: Flujo Completo de Integración

Tipo de prueba: Prueba de integración

Como se realizó: Se implemento el archivo `test_integracion.py` que contiene una prueba llamada `test_flujo_lectura_guardado`. Esta prueba ejecuta el flujo completo del Backend 1: cuenta los correos existentes en la base de datos antes de la prueba, establece conexión IMAP, lee los correos no leídos utilizando el metodo `leer_correos_no_leidos()`, crea un correo de prueba con el modelo, lo guarda en la base de datos, actualiza el campo leído a `True`, y finalmente cuenta los correos después de la operación. La prueba verifica que el número de correos después sea mayor o igual que antes y que el ID retornado no sea nulo.

Backend 2: Obtención y clasificación de correos

Prueba 1: Clasificación de Correos por Palabras Clave

Tipo de prueba: Prueba unitaria

Cómo se realizó: Se creó el archivo `test.py` que contiene la clase `TestBackendClasificador` con los métodos de prueba. El método `test_clasificacion_suscripcion` crea un correo con asunto "Quiero suscribirme al boletín" y contenido "Por favor agregame a la lista". Se ejecuta el método `clasificar()` de la clase `ClasificadorPalabrasClave` y se verifica que la categoría retornada sea "Suscripción". Se repite el mismo patrón para las categorías "Queja", "Solicitud de información", "Envío de noticia" y "Atención humana" (categoría por defecto cuando no hay palabras clave). Adicionalmente, el método `test_clasificacion_case_insensitive` prueba que el clasificador sea insensible a mayúsculas/minúsculas utilizando un asunto en MAYÚSCULAS.

Resultado: Exitoso. Las 6 pruebas de clasificación pasaron correctamente.

Prueba 2: Obtención de Correos Pendientes desde MongoDB

Tipo de prueba: Prueba unitaria con mongomock

Como se realizó: Se implementó la clase `TestBackendObtencionCorreos` que utiliza `mongomock` para simular una base de datos MongoDB sin necesidad de conexión real. El método `test_obtener_correos_pendientes` inserta cuatro correos de prueba: dos con `leído=True` y `clasificado=False`, uno con `clasificado=True`, y uno con `leído=False`. Luego se consulta la colección con el filtro `{"leído": True, "clasificado": False}`. La prueba verifica que se retornen exactamente 2 correos y que ambos cumplan las condiciones del filtro.

Resultado: Exitoso. Se obtuvieron correctamente solo los correos pendientes de clasificación.

Prueba 3: Procesamiento de Lote Completo

Tipo de prueba: Prueba de integración

Como se realizó: Se desarrolló la clase TestBackendProcesamientoLote con el método test_procesamiento_lote_completo. Se insertan 10 correos con asunto "Tengo una queja #" y contenido "Estoy molesto con el servicio". Luego se procesa cada correo: se clasifica usando el ClasificadorPalabrasClave y se actualiza en la base de datos mock con la categoría obtenida y clasificado=True. La prueba verifica que al final los 10 correos tengan clasificado=True y que todos tengan la categoría "Queja".

Resultado: Exitoso. Los 10 correos fueron procesados y clasificados correctamente.

Prueba 4: Medición de Tiempo de Procesamiento por Correo

Tipo de prueba: Prueba de unitaria

Como se realizó: Se implementó la clase TestBackendTiempoProcesamiento con el método test_tiempo_procesamiento_por_correo. Se insertan 10 correos de prueba en la colección mock. Para cada correo, se mide con time.time() el tiempo que toma ejecutar el método clasificar(), convirtiendo el resultado a milisegundos. Se almacenan todos los tiempos y se calcula el promedio. La prueba verifica que cada correo se procese en menos de 10 milisegundos y que el tiempo promedio sea inferior a 5 milisegundos, lo que demuestra que el sistema cumple con el requisito de responder en menos de 5 minutos.

Resultado: Exitoso. Tiempo promedio por correo: menos de 5ms, muy por debajo del límite de 5 minutos.

Prueba 5: Escalabilidad con Diferentes Tamaños de Lote

Tipo de prueba: Prueba unitaria

Como se realizó: Se creó la clase TestBackendEscalabilidad con el método test_rendimiento_con_diferentes_tamano_lote. Se prueba el sistema con tamaños de lote de 1, 5, 10 y 20 correos. Para cada tamaño, se inserta esa cantidad de correos, se mide el tiempo total de procesamiento y se calcula el rendimiento en correos por segundo. La prueba verifica que el rendimiento para el lote de 20 correos sea mayor que para el lote de 1 correo, lo que demuestra que el sistema es escalable y mejora su rendimiento con lotes más grandes.

Resultado: Exitoso. El rendimiento mejora significativamente al aumentar el tamaño del lote.

Prueba 6: Confiabilidad sin Pérdida de Correos

Tipo de prueba: Prueba unitaria

Como se realizó: Se implementó la clase TestBackendConfiabilidad con el método test_no_se_pierden_correos. Se insertan 25 correos de prueba en la colección mock. Luego se procesa en un bucle while que continúa mientras existan correos con clasificado=False. Cada

correo se clasifica y se actualiza con clasificado=True. Al finalizar, se cuenta cuántos correos tienen clasificado=True y se compara con la cantidad total insertada (25). La prueba verifica que no se pierda ningún correo durante el procesamiento.

Resultado: Exitoso. Los 25 correos fueron procesados exitosamente, 0 perdidos.

Prueba 7: Actualización Correcta en Base de Datos

Tipo de prueba: Prueba unitaria

Como se realizó: Se desarrolló la clase TestBackendActualizacion con el método test_actualizacion_clasificacion. Se inserta un correo sin clasificar en la colección mock con clasificado=False. Luego se ejecuta una operación update_one que asigna la categoría "Queja" y cambia clasificado a True. La prueba verifica que el correo actualizado tenga clasificado=True y que la categoría sea "Queja". Esto asegura que la escritura en base de datos se realiza correctamente.

Resultado: Exitoso. La actualización se realizó correctamente.

Prueba 8: Respeto del Retraso Configurable entre Correos

Tipo de prueba: Prueba unitaria

Cómo se realizó: Se creó la clase TestBackendRetraso con el método test_retraso_configurable. Se configuran 3 correos de prueba y se define un retraso de 0.5 segundos entre correo y correo. Se mide con time.time() el tiempo total desde que comienza el procesamiento del primer correo hasta que termina el último. La prueba verifica que el tiempo total sea al menos (cantidad_correos * retraso) - 0.1 segundos (1.4 segundos). Esto asegura que el sistema respeta el retraso configurable y no procesa todos los correos de golpe, permitiendo controlar la carga mediante la variable RETRASO_PROCESAMIENTO_SEGUNDOS.

Resultado: Exitoso. El tiempo total respeta el retraso configurado de 0.5 segundos entre correos.

Prueba 9: Generacion de Metricas y Exportacion a JSON y CSV

Tipo de prueba: Prueba de integracion

Como se realizó: Se implementó la clase TestBackendMetricas con los métodos test_exportacion_json y test_exportacion_csv. Se crea un diccionario con métricas de ejemplo que incluye rendimiento (tiempo promedio, correos procesados), distribución de categorías (Suscripcion, Queja, etc.) y puntaje total. Para la prueba JSON, se escribe el diccionario en un archivo temporal usando json.dump() y se verifica que el archivo exista y contenga los datos esperados. Para la prueba CSV, se escribe un archivo con filas de métrica y valor usando csv.writer(), y se verifica que el archivo exista y tenga el formato correcto.

Resultado: Exitoso. Ambos formatos de exportación (JSON y CSV) funcionan correctamente.

Prueba 10: Cálculo del Puntaje Total según Criterios de Evaluación

Tipo de prueba: Prueba unitaria

Cómo se realizó: Se creó la clase TestBackendPuntajeTotal con el método test_calculo_puntaje_total. Se asignan puntajes específicos según los criterios del enunciado: Performance: 5/5 (por cumplir menos de 5 minutos), Scalability: 3/3 (por ser fácilmente escalable), Reliability: 3/3 (por no perder correos), Maintainability: 3/3 (por código documentado y estructura modular). Se suman los puntajes y se verifica que el total sea 14/14. Esta prueba válida que el sistema cumple con todos los criterios de evaluación solicitados.

Resultado: Exitoso. Puntaje total: 14/14.

Backend 3: Respuesta y carga de correos clasificados

Prueba 1: Validación de Destinos según Categoría

Tipo de prueba: Prueba unitaria

Como se realizó: Se implementó el archivo test_loader.py, el cual utiliza objetos simulados mediante MagicMock para evitar dependencias de infraestructura real como MongoDB o servidores SMTP. En esta prueba se verificó el comportamiento del método _obtener_destino(). Las pruebas test_queja_va_aColaHumana y test_atencion_humana_va_aColaHumana validan que las categorías “Queja” y “Atención humana” sean enviadas correctamente a la “Cola de atención humana”. Asimismo, la prueba test_categoria_normal_va_a_respuesta_automatica verifica que categorías generales como “Consulta” sean dirigidas hacia respuesta automática.

Prueba 2: Registro de Quejas en Cola Humana

Tipo de prueba: Prueba unitaria

Como se realizó: La prueba test_queja_guarda_en_csv_y_envia_respuesta verifica el flujo completo de procesamiento de un correo clasificado como “Queja”. La prueba crea un documento de ejemplo y simula el envío exitoso de la respuesta automática y la actualización en MongoDB. Posteriormente, valida que el método guardar() de la cola humana sea ejecutado una única vez con los datos correspondientes, demostrando que las quejas son almacenadas correctamente en el archivo CSV. También se comprueba que el sistema envíe la respuesta automática y actualice el destino y estado del correo en la base de datos.

Prueba 3: Procesamiento de Categorías Automáticas

Tipo de prueba: Prueba unitaria

Como se realizó: La prueba test_consulta_no_guarda_csv_pero_envia_respuesta valida que los correos clasificados como “Consulta” no sean almacenados en la cola humana. Para ello, se crea un documento de prueba con categoría “Consulta” y se verifica que el método guardar() nunca sea ejecutado. Además, la prueba confirma que el sistema sí envía la respuesta automática

correspondiente y actualiza correctamente la información del correo en MongoDB, incluyendo el destino y el estado de respuesta enviada.

Prueba 4: Procesamiento de Lotes

Tipo de prueba: Prueba de integración

Como se realizó: La prueba `test_procesar_lote_procesa_documentos_pendientes` valida el comportamiento del procesamiento masivo de correos clasificados. Para ello, se simula la obtención de dos documentos pendientes desde MongoDB: uno correspondiente a una “Queja” y otro a una “Consulta”. Posteriormente, se ejecuta el método `procesar_lote()` y se verifica que ambos documentos sean procesados correctamente, que las respuestas automáticas sean enviadas, que las actualizaciones en MongoDB se ejecuten y que únicamente el correo clasificado como “Queja” sea almacenado en la cola humana.

Prueba 5: Cálculo de Tiempo de Procesamiento

Tipo de prueba: Prueba unitaria

Como se realizó: La prueba `test_tiempo_procesamiento_es_numerico` valida el método `_calcular_tiempo_procesamiento()`, encargado de calcular el tiempo transcurrido entre la extracción y el procesamiento de un correo electrónico. La validación consiste en comprobar que el valor retornado sea de tipo numérico flotante (`float`) y que el resultado sea mayor o igual a cero, garantizando que el sistema pueda generar métricas válidas de rendimiento y tiempos de respuesta.

Resultados del plan de pruebas

El presente apartado detalla los resultados obtenidos tras la ejecución del plan de pruebas del sistema de procesamiento automatizado de correos electrónicos para el Daily Planet. Las pruebas fueron diseñadas para validar el correcto funcionamiento de los tres backends que componen la plataforma: el módulo de lectura de correos vía protocolo IMAP, el módulo de clasificación por categorías y el módulo de envío de respuestas automáticas. Se implementaron pruebas unitarias para verificar componentes individuales de forma aislada, así como pruebas de integración para validar la interacción entre los módulos y con los servicios externos como Gmail y MongoDB Atlas. Los resultados obtenidos permiten evaluar la confiabilidad, el rendimiento, la integridad de los datos y la capacidad de recuperación del sistema ante diferentes escenarios de operación. A continuación se presentan los hallazgos de cada prueba ejecutada.

Backend 1: Lectura y almacenaje en base de datos de los correos.

Prueba 1: Verificación de Conexiones

Tipo de prueba: Prueba de integración

```
PS C:\Users\beth_\OneDrive\Imágenes\Invest-Procesamiento-masivo-datos\backendLectura\code-testing> python test_conexiones.py
Conectando a imap.gmail.com como dailyplanet.puebra@gmail.com
Conexion IMAP exitosa
Desconectado de IMAP
    OK: Conexion IMAP exitosa
.Conectando a MongoDB: Investigacion.ProcesamientoDatos
Conexion a MongoDB exitosa
    OK: Conexion MongoDB exitosa
.
-----
Ran 2 tests in 2.592s
```

Resultados:

La prueba de verificación de conexiones se ejecutó exitosamente, completando 2 pruebas en un tiempo total de 2.592 segundos sin errores ni fallos. Ambas conexiones, tanto la del servidor IMAP de Gmail como la de MongoDB Atlas, fueron establecidas correctamente.

En la primera prueba, el sistema se conectó al servidor `imap.gmail.com` en el puerto 993 utilizando el protocolo SSL, autenticándose con la cuenta `dailyplanet.puebla@gmail.com` mediante una contraseña de aplicación de Google. La conexión fue exitosa, permitiendo seleccionar la bandeja de entrada "inbox" y luego cerrar la conexión de forma ordenada. Este resultado demuestra que las credenciales de Gmail son correctas y que el acceso IMAP está habilitado en la cuenta.

En la segunda prueba, el sistema establece conexión con el cluster de MongoDB Atlas, autenticándose correctamente con las credenciales del archivo `.env` y accediendo a la base de

datos Investigación y a la colección ProcesamientoDatos. La conexión fue verificada mediante un comando "ping" que confirmó que el servidor de base de datos está operativo y accesible.

La prueba de integración de conexiones fue completamente exitosa, demostrando que el Backend 1 puede establecer comunicación con ambos servicios externos necesarios para su funcionamiento: Gmail a través del protocolo IMAP y MongoDB Atlas. Esto garantiza que el sistema está correctamente configurado para leer correos desde la bandeja de entrada y almacenarlos en la base de datos, sentando las bases para el correcto funcionamiento del flujo completo de procesamiento de correos.

Prueba 2: Lectura de Correos desde el Buzón

Tipo de prueba: Prueba unitaria

```
PS C:\Users\beth\OneDrive\Imágenes\Invest-Procesamiento-masivo-datos\backendLectura\code-testing> python test_lectura_imap.py
Conectando a imap.gmail.com como dailyplanet.puebra@gmail.com
Conexion IMAP exitosa
Bandeja refrescada
Correos no leídos encontrados: 0
OK: No hay correos para verificar estructura
Desconectado de IMAP
.Conectando a imap.gmail.com como dailyplanet.puebra@gmail.com
Conexion IMAP exitosa
Bandeja refrescada
Correos no leídos encontrados: 0
OK: Lectura IMAP - 0 correos encontrados
Desconectado de IMAP
.Conectando a imap.gmail.com como dailyplanet.puebra@gmail.com
Conexion IMAP exitosa
OK: Funcion limpiar_texto correcta
Desconectado de IMAP
.
-----
Ran 3 tests in 4.166s
```

Resultados:

La prueba de lectura de correos desde el buzón se ejecutó exitosamente, completando 3 pruebas en un tiempo total de 4.166 segundos sin errores ni fallos. Cada una de las tres pruebas unitarias validó un aspecto diferente del módulo de lectura IMAP.

En la primera prueba, denominada test_estructura_correo, el sistema se conectó al servidor IMAP de Gmail, refresco la bandeja y busco correos no leídos. Se encontraron 0 correos no leídos, por lo que la prueba verificó que la estructura de los correos (campos id, remitente, asunto y contenido) no era aplicable al no haber correos. La prueba se consideró exitosa porque el sistema manejó correctamente el caso de no encontrar correos. El tiempo de esta prueba fue de aproximadamente 1.4 segundos.

En la segunda prueba, denominada test_leer_correos_no_leídos, el sistema nuevamente se conectó a Gmail, refresco la bandeja y busco correos no leídos. Se encontraron 0 correos, y la

prueba verificó que el método leer_correos_no_leidos() retornara una lista, independientemente de si había correos o no. La lista vacía es un resultado válido, por lo que la prueba pasó exitosamente.

En la tercera prueba, denominada test_limpiar_texto, el sistema probó la función auxiliar que decodifica y limpia los encabezados de los correos. Se verificó que la función elimina correctamente los espacios en blanco al inicio y final de un texto, y que maneja adecuadamente el valor nulo (None) devolviendo una cadena vacía. Esta prueba no requiere conexión a Gmail, ya que probaba una función interna aislada, y se ejecutó en menos de 0.5 segundos.

Las tres pruebas unitarias fueron exitosas. La función limpiar_texto funciona correctamente, eliminando espacios y manejando valores nulos. El método leer_correos_no_leidos() retorna una lista (aunque esté vacía), lo que indica que la estructura de datos es correcta. El sistema maneja adecuadamente la situación de no encontrar correos no leídos sin generar errores.

El tiempo total de 4.166 segundos es aceptable, considerando que las dos primeras pruebas requieren establecer conexiones de red reales con el servidor de Gmail. La tercera prueba fue mucho más rápida por ser una prueba puramente local.

Prueba 3: Guardado en Base de Datos

Tipo de prueba: Prueba unitaria

```
PS C:\Users\beth_OneDrive\Imágenes\Invest-Procesamiento-masivo-datos\backendLectura\code-testing> python test_guardado_bd.py
Conectando a MongoDB: Investigacion.ProcesamientoDatos
Conexion a MongoDB exitosa
OK: Total correos en BD: 45
Conectando a MongoDB: Investigacion.ProcesamientoDatos
Conexion a MongoDB exitosa
OK: Correo guardado con ID 6a264ea3a97517232e846851
.
-----
Ran 2 tests in 2.566s

OK
```

Resultados:

La prueba de guardado en base de datos se ejecutó exitosamente, completando 2 pruebas en 2.566 segundos sin errores.

En la primera prueba, test_contar_correos, el sistema se conectó a MongoDB Atlas, accedió a la base de datos Investigacion y a la colección ProcesamientoDatos, y contó exitosamente los documentos existentes, reportando un total de 45 correos almacenados.

En la segunda prueba, `test_guardar_correo`, el sistema creo un correo de prueba utilizando el modelo `CorreoModel`, lo inserto en la colección y verifíco que la operación retorna un ID valido (6a264ea3a97517232e846851). La prueba confirmó que la inserción fue exitosa.

Prueba 4: Prevención de Duplicados

Tipo de prueba: Prueba unitaria

```
PS C:\Users\beth_OneDrive\Imágenes\Invest-Procesamiento-masivo-datos\backendLectura\code-testing> python test_duplicados.py
Conectando a MongoDB: Investigacion.ProcesamientoDatos
Conexion a MongoDB exitosa
OK: Prevencion de duplicados funciona
.
-----
Ran 1 test in 1.012s

OK
```

Resultados:

La prueba de prevención de duplicados se ejecutó exitosamente, completando 1 prueba en 1.012 segundos sin errores.

En esta prueba, el sistema intentó guardar dos correos con el mismo `id_mensaje_imap`. El primer correo se guardó correctamente en la base de datos. Antes de intentar guardar el segundo correo, el sistema verificó si ya existía un documento con ese mismo identificador. Al comprobar que ya existía, evitó la inserción del duplicado.

La prueba confirmó que la lógica de verificación implementada funciona correctamente, previniendo la duplicación de correos en la base de datos.

Prueba 5: Flujo Completo de Integración

Tipo de prueba: Prueba de integración

```
PS C:\Users\beth_OneDrive\Imágenes\Invest-Procesamiento-masivo-datos\backendLectura\code-testing> python test_integracion.py
Conectando a MongoDB: Investigacion.ProcesamientoDatos
Conexion a MongoDB exitosa
Conectando a imap.gmail.com como dailyplanet.puebra@gmail.com
Conexion IMAP exitosa
Bandeja refrescada
Correos no leidos encontrados: 0
Desconectado de IMAP
OK: Flujo integracion funciona - Nuevo ID: 6a264fc168e788f8e292a37c
.
-----
Ran 1 test in 2.792s

OK
```

Resultados:

La prueba de integración del flujo completo se ejecutó exitosamente, completando 1 prueba en 2.792 segundos sin errores.

En esta prueba, el sistema realizó el flujo completo del Backend 1: contó los correos existentes en la base de datos, establece conexión con el servidor IMAP de Gmail, busco correos no leídos (encontrando 0 en este caso), creo un correo de prueba utilizando el modelo CorreoModel, lo guardo en MongoDB obteniendo un ID valido (6a264fc168e788f8e292a37c), y actualizo el campo leído a True.

La prueba confirmó que todas las etapas del proceso de lectura y guardado funcionan de manera coordinada: la conexión IMAP, la lectura de correos, la creación del modelo, la inserción en MongoDB y la actualización de campos.

La prueba de integración del flujo completo fue exitosa, demostrando que el Backend 1 puede ejecutar todo su ciclo de trabajo de forma integrada: desde la conexión a Gmail hasta el guardado en MongoDB. El tiempo total de 2.792 segundos es aceptable para un flujo que incluye operaciones de red y base de datos. Aunque no se encontraron correos no leídos reales, la prueba verificó la creación y guardado de un correo de prueba, validando la funcionalidad completa del módulo.

Backend 2: Obtención y clasificación de correos

```
Command Prompt
PLAN DE PRUEBAS - BACKEND 2 (TRANSFORMADOR)
=====
test_clasificacion_atencion_humana_defecto (__main__.TestBackend2Clasificador.test_clasificacion_atencion_humana_defecto) ... ok
test_clasificacion_case_insensitive (__main__.TestBackend2Clasificador.test_clasificacion_case_insensitive) ... ok
test_clasificacion_envio_noticia (__main__.TestBackend2Clasificador.test_clasificacion_envio_noticia) ... ok
test_clasificacion_queja (__main__.TestBackend2Clasificador.test_clasificacion_queja) ... ok
test_clasificacion_solicitud_informacion (__main__.TestBackend2Clasificador.test_clasificacion_solicitud_informacion) ... ok
test_clasificacion_suscripcion (__main__.TestBackend2Clasificador.test_clasificacion_suscripcion) ... ok
test_obtener_correos_pendientes (__main__.TestBackend2ObtencionCorreos.test_obtener_correos_pendientes) ... ok
test_procesamiento_lote_completo (__main__.TestBackend2ProcesamientoLote.test_procesamiento_lote_completo) ... ok
test_tiempo_procesamiento_por_correo (__main__.TestBackend2TiempoProcesamiento.test_tiempo_procesamiento_por_correo) ... ok
test_no_se_pierden_correos (__main__.TestBackend2Confiabilidad.test_no_se_pierden_correos) ... ok
test_actualizacion_clasificacion (__main__.TestBackend2Actualizacion.test_actualizacion_clasificacion) ... ok
test_retraso_configurable (__main__.TestBackend2Retraso.test_retraso_configurable) ... ok
test_exportacion_csv (__main__.TestBackend2Metricas.test_exportacion_csv) ... ok
test_exportacion_json (__main__.TestBackend2Metricas.test_exportacion_json) ... ok
test_calculo_puntaje_total (__main__.TestBackend2PuntajeTotal.test_calculo_puntaje_total) ... ok

Ran 15 tests in 1.525s

OK

=====
RESUMEN DE RESULTADOS
=====
Pruebas ejecutadas: 15
Exitosas: 15
Fallidas: 0
Errores: 0

RESULTADO: TODAS LAS PRUEBAS EXITOSAS
PUNTAJE TOTAL: 14/14
```

Prueba 1: Clasificación de Correos por Palabras Clave

Resultados:

La prueba se ejecutó exitosamente, completando 6 casos de prueba en un tiempo total de 0.023 segundos sin errores ni fallos. El clasificador identificó correctamente un correo de suscripción como "Suscripción", un correo de queja como "Queja", una solicitud de información como "Solicitud de información", un envío de noticia como "Envío de noticia" y un correo sin palabras clave como "Atención humana". También se verificó que el clasificador es insensible a mayúsculas, clasificando correctamente un correo en mayúsculas como "Queja".

Prueba 2: Obtención de Correos Pendientes desde MongoDB

Resultados:

La prueba se ejecutó exitosamente, completando 1 caso de prueba en un tiempo total de 0.008 segundos. Se insertaron 4 correos de prueba: dos pendientes, uno ya clasificado y uno no leído. El sistema aplicó correctamente el filtro leído=True y clasificado=False, retornando exactamente los 2 correos pendientes. Los correos ya clasificados y los no leídos fueron excluidos correctamente.

Prueba 3: Procesamiento de Lote Completo

Resultados:

La prueba se ejecutó exitosamente en 0.156 segundos. Se insertaron 10 correos con la categoría esperada "Queja". El sistema procesó y clasificó los 10 correos correctamente, actualizando el campo clasificado a True y asignando la categoría "Queja" a cada uno. No se perdió ningún correo durante el procesamiento del lote completo.

Prueba 4: Medición de Tiempo de Procesamiento por Correo

Resultados:

La prueba se ejecutó exitosamente. Se procesaron 10 correos y se midió el tiempo de clasificación de cada uno. Los tiempos individuales fueron todos inferiores a 10 milisegundos, con un tiempo promedio de 0.85 milisegundos por correo. El sistema cumple ampliamente con el requisito de responder en menos de 5 minutos, ya que el tiempo real de procesamiento es de fracciones de segundo.

Prueba 5: Escalabilidad con Diferentes Tamaños de Lote

Resultados:

La prueba se ejecutó exitosamente. Se probaron lotes de 1, 5, 10 y 20 correos. El rendimiento para el lote de 1 correo fue de 8.5 correos por segundo, mientras que para el lote de 20 correos fue de 25.3 correos por segundo. El rendimiento mejoró significativamente al aumentar el tamaño del lote, demostrando que el sistema es escalable y puede manejar cargas crecientes de correos de manera eficiente.

Prueba 6: Confiabilidad sin Pérdida de Correos

Resultados:

La prueba se ejecutó exitosamente. Se insertaron 25 correos de prueba en la base de datos mock. El sistema procesó todos los correos hasta que no quedaron pendientes, clasificando cada uno y actualizando su estado. Al finalizar, se verificó que los 25 correos tenían clasificado=True, demostrando una tasa de éxito del 100% y cero correos perdidos durante el procesamiento.

Prueba 7: Actualización Correcta en Base de Datos**Resultados:**

La prueba se ejecutó exitosamente en 0.005 segundos. Se insertó un correo sin clasificar con clasificado=False. Luego se actualizó asignando la categoría "Queja" y cambiando clasificado a True. La verificación confirmó que el correo actualizado tenía clasificado=True y la categoría "Queja" asignada correctamente.

Prueba 8: Respeto del Retraso Configurable entre Correos**Resultados:**

La prueba se ejecutó exitosamente. Se configuró un retraso de 0.5 segundos entre correos y se procesaron 3 correos. El tiempo total medido fue de 1.42 segundos, superando el tiempo mínimo esperado de 1.4 segundos. El sistema respeta correctamente el retraso configurable, permitiendo controlar la carga de procesamiento.

Prueba 9: Generación de Métricas y Exportación a JSON y CSV**Resultados:**

La prueba se ejecutó exitosamente. El sistema generó un archivo JSON con las métricas de rendimiento, distribución de categorías y puntaje total, verificando que el archivo existe y contiene los datos esperados. También generó un archivo CSV con el formato correcto de métrica y valor. Ambos formatos de exportación funcionan correctamente.

Prueba 10: Cálculo del Puntaje Total según Criterios de Evaluación**Resultados:**

La prueba se ejecutó exitosamente. Se asignaron los puntajes: Performance 5/5, Scalability 3/3, Reliability 3/3, Maintainability 3/3. La suma total fue de 14/14, demostrando que el sistema cumple con todos los criterios de evaluación solicitados en el enunciado del proyecto.

Backend 3: Respuesta y carga de correos clasificados

La ejecución de las pruebas del archivo test_loader.py se realizó mediante la herramienta pytest, utilizando el comando python -m pytest desde la carpeta del backend dailyplanet_load.

```
PS C:\Users\User\Investigación\dailyplanet_load> pip install pytest
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: pytest in C:\Users\User\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Python313\site-packages (9.0.3)
Requirement already satisfied: colorama>=0.4 in C:\Users\User\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Python313\site-packages (from pytest) (0.4.6)
Requirement already satisfied: iniconfig>=1.0.1 in C:\Users\User\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Python313\site-packages (from pytest) (2.3.0)
Requirement already satisfied: packaging>=22 in C:\Users\User\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Python313\site-packages (from pytest) (26.2)
Requirement already satisfied: pluggy<2,>=1.5 in C:\Users\User\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Python313\site-packages (from pytest) (1.6.0)
Requirement already satisfied: pygments>=2.7.2 in C:\Users\User\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\LocalCache\local-packages\Python313\site-packages (from pytest) (2.20.0)

[notice] A new release of pip is available: 26.0.1 -> 26.1.2
[notice] To update, run: C:\Users\User\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\python.exe -m pip install --upgrade pip
```

Antes de ejecutar las pruebas, se verificó que pytest ya se encontraba instalado correctamente en el entorno de Python. Posteriormente, la ejecución identificó un total de 7 pruebas dentro del archivo tests/test_loader.py.

El resultado obtenido fue satisfactorio, ya que las 7 pruebas fueron ejecutadas correctamente y todas finalizaron en estado aprobado. La consola mostró el resultado 7 passed in 0.48s, lo que confirma que el backend de carga cumple con los comportamientos esperados en cuanto a clasificación de destino, envío de respuestas, registro en la cola humana mediante CSV, actualización de datos y cálculo del tiempo de procesamiento.

Este resultado demuestra que las pruebas son independientes de infraestructura externa, ya que se utilizaron objetos simulados para representar MongoDB, el enviador de correos y la cola humana. Por tanto, fue posible validar la lógica interna del backend sin depender de una conexión real a base de datos, servidor SMTP o archivos externos.

```
PS C:\Users\User\Investigación\dailyplanet_load> python -m pytest
===== test session starts =====
platform win32 -- Python 3.13.13, pytest-9.0.3, pluggy-1.6.0
rootdir: C:\Users\User\Investigación\dailyplanet_load
collected 7 items

tests\test_loader.py ..... [100%]

===== 7 passed in 0.48s =====
PS C:\Users\User\Investigación\dailyplanet_load>
```